

Sliding Window

Sliding Window is an optimization technique used to convert $O(N^2)$ or $O(N^3)$ problems into $O(N)$ or $O(N \log K)$ by maintaining a "window" of the data that shifts as you iterate.

Fixed Size Window (Frequency Mapping)

Theory: Used when the window length K is constant. You move the window by adding one element from the right and removing one from the left.

HFT Tip: Avoid HashMap. Use a fixed-size `int[]` array. For lowercase letters, `int[26]` is faster; for all ASCII, use `int[128]`. This reduces heap allocation and improves cache locality.

Question: Permutation in String (Check if a permutation of string p exists in string s).

Logic: Maintain a frequency count of characters in p . Slide a window of size `p.length()` across s . If the frequency counts match, you found a permutation.

```
public boolean checkInclusion(String p, String s) {
    if (p.length() > s.length()) return false;
    int[] pCount = new int[26];
    int[] sCount = new int[26];

    // Build initial window
    for (int i = 0; i < p.length(); i++) {
        pCount[p.charAt(i) - 'a']++;
        sCount[s.charAt(i) - 'a']++;
    }

    for (int i = p.length(); i < s.length(); i++) {
        if (Arrays.equals(pCount, sCount)) return true;
        // Slide: Add right character, remove leftmost character
        sCount[s.charAt(i) - 'a']++;
        sCount[s.charAt(i - p.length()) - 'a']--;
    }
    return Arrays.equals(pCount, sCount);
}
```

Variable Size Window (The "Shrink" Template)

Theory: The window size expands until a condition is violated, then shrinks from the left until the condition is valid again.

Key Property: Monotonicity. As you expand right, the condition (e.g., sum) only increases. As you move left, it only decreases.

Question: Longest Substring with at most K Distinct Characters.

Logic: Use a HashMap to store counts. Expand right. If the map size exceeds K, move left and decrement counts until the map size is $\leq K$.

```
public int lengthOfLongestSubstringKDistinct(String s, int k) {
    int[] counts = new int[128]; // ASCII frequency
    int left = 0, distinct = 0, maxLen = 0;

    for (int right = 0; right < s.length(); right++) {
        if (counts[s.charAt(right)] == 0) distinct++;
        counts[s.charAt(right)]++;

        while (distinct > k) {
            counts[s.charAt(left)]--;
            if (counts[s.charAt(left)] == 0) distinct--;
            left++;
        }
        maxLen = Math.max(maxLen, right - left + 1);
    }
    return maxLen;
}
```

Monotonic Queue (Max/Min Variant)

Theory: Standard sliding window cannot give you the max/min of the window in $O(1)$ without extra help. A Deque maintains indices of elements in a strictly decreasing (for Max) or increasing (for Min) order.

Constraint: $O(N)$ time because each index is pushed/popped exactly once.

Question: Sliding Window Maximum.

Logic: Before adding index i , remove all indices from the back of the Deque whose values are less than $\text{nums}[i]$, as they can never be the maximum for any future window.

```

public int[] maxSlidingWindow(int[] nums, int k) {
    int n = nums.length;
    int[] res = new int[n - k + 1];
    Deque<Integer> dq = new ArrayDeque<>(); // Indices

    for (int i = 0; i < n; i++) {
        // 1. Remove out-of-bounds index
        if (!dq.isEmpty() && dq.peekFirst() == i - k) dq.pollFirst();

        // 2. Maintain monotonic decreasing: remove smaller elements
        while (!dq.isEmpty() && nums[dq.peekLast()] < nums[i]) dq.pollLast();

        dq.addLast(i);

        // 3. Peek front for current max
        if (i >= k - 1) res[i - k + 1] = nums[dq.peekFirst()];
    }
    return res;
}

```

Two Heaps (Sliding Window Median)

Theory: To find the median, you need the middle element(s). Maintaining a sorted window is $O(K \log K)$ per slide. Using two heaps (Max-Heap for the left half, Min-Heap for the right half) reduces this to $O(\log K)$ per slide.

HFT Optimization: In Java, `PriorityQueue.remove(Object)` is $O(K)$. For true $O(\log K)$, use two `TreeMaps` (to act as indexed heaps) or a custom Balance Tree.

Question: Sliding Window Median.

Logic: Keep the size of `leftHeap` and `rightHeap` balanced. The median is either the top of `leftHeap` or the average of both tops.

```

public double[] medianSlidingWindow(int[] nums, int k) {
    public double[] medianSlidingWindow(int[] nums, int k) {
        double[] res = new double[nums.length - k + 1];
        TreeMap<Integer, Integer> left = new TreeMap<>(), right = new TreeMap<>();
        int leftSize = 0, rightSize = 0;

        for (int i = 0; i < nums.length; i++) {
            // 1. Add new element
            if (leftSize <= rightSize) {
                right.put(nums[i], right.getDefault(nums[i], 0) + 1);
                int first = right.firstKey();
                remove(right, first);
                left.put(first, left.getDefault(first, 0) + 1);
                leftSize++;
            } else {
                left.put(nums[i], left.getDefault(nums[i], 0) + 1);
                int last = left.lastKey();
                remove(left, last);
                right.put(last, right.getDefault(last, 0) + 1);
                rightSize++;
            }

            // 2. Remove out-of-window element
            if (i >= k) {
                int toRemove = nums[i - k];
                if (left.containsKey(toRemove)) {
                    remove(left, toRemove);
                    leftSize--;
                } else {
                    remove(right, toRemove);
                    rightSize--;
                }
            }

            // 3. Get Median
            if (i >= k - 1) {
                if (k % 2 == 1) res[i - k + 1] = (double) left.lastKey();
                else res[i - k + 1] = ((double) left.lastKey() + right.firstKey()) / 2.
            }
        }
        return res;
    }
}

```

```
private void remove(TreeMap<Integer, Integer> map, int val) {
    map.put(val, map.get(val) - 1);
    if (map.get(val) == 0) map.remove(val);
}
```

The "At Most to Exactly" Transformation

Theory: Problems asking for "Exactly K" are often non-monotonic (the count doesn't move linearly). We transform the requirement into two monotonic problems.

Formula: $\text{Exactly}(K) = \text{AtMost}(K) - \text{AtMost}(K-1)$.

Question: Subarrays with K Different Integers.

Logic: Write a helper function `atMost(k)` that counts subarrays with $\leq K$ distinct numbers. The number of subarrays ending at right is `right - left + 1`.

```
public int subarraysWithKDistinct(int[] nums, int k) {
    return atMost(nums, k) - atMost(nums, k - 1);
}

private int atMost(int[] nums, int k) {
    Map<Integer, Integer> count = new HashMap<>();
    int left = 0, res = 0;
    for (int right = 0; right < nums.length; right++) {
        if (count.containsKey(nums[right]) == 0) k--;
        count.put(nums[right], count.getOrDefault(nums[right], 0) + 1);
        while (k < 0) {
            count.put(nums[left], count.get(nums[left]) - 1);
            if (count.get(nums[left]) == 0) k++;
            left++;
        }
        res += right - left + 1;
    }
    return res;
}
```

The "Non-Shrinkable" Optimized Window

Theory: In "Longest Substring" problems, you don't actually need to shrink the window back to a valid state. You just ensure it never gets smaller than the maximum found so far.

Why use this? It's a "Global Max" strategy often seen in competitive programming to squeeze out performance.

Question: Longest Repeating Character Replacement.

Logic: Keep track of the frequency of the most frequent character in the current window (maxFreq). If $(\text{windowSize} - \text{maxFreq}) > k$, the window is invalid. Instead of a while loop, use a single if to shift the window.

```
public int characterReplacement(String s, int k) {
    int[] count = new int[26];
    int left = 0, maxFreq = 0, maxLen = 0;
    for (int right = 0; right < s.length(); right++) {
        maxFreq = Math.max(maxFreq, ++count[s.charAt(right) - 'a']);
        // If window is invalid, just shift it
        if (right - left + 1 - maxFreq > k) {
            count[s.charAt(left) - 'a']--;
            left++;
        }
        maxLen = Math.max(maxLen, right - left + 1);
    }
    return maxLen;
}
```

Question	Logic	Key DS
"Anagram", "Permutation"	Fixed Size	int[26]
"Smallest/Shortest", "Sum $\geq K$ "	Variable Shrink	while loop + sum
"Max/Min in every window"	Monotonic	Deque<Integer> (indices)
"Median in window"	Two Heaps	TreeMap (for $O(\log K)$ remove)
"Exactly K elements"	Difference	atMost(K) - atMost(K-1)
"Longest" with replacement	Non-Shrink	if instead of while

Final Revision Note : If the problem involves negative numbers and asks for a sum, Sliding Window is likely impossible because adding an element could decrease the sum (violating monotonicity). In that case, use Prefix Sums.

Prefix Sum

Prefix Sum is the "inverse" of the Difference Array. It is the primary tool for problems involving negative numbers or cases where you need to check the "history" of the array.

1. Basic 1D Prefix Sum (Range Sum Query)

Theory: Pre-calculate an array P where $P[i]$ is the sum of $nums[0...i-1]$.

Formula: $Sum(i,j) = P[j+1] - P[i]$

HFT Tip: Always use `long[]` for prefix sums to prevent integer overflow when processing large datasets or high-frequency ticks.

Question: Range Sum Query - Immutable.

Logic: By pre-calculating the sum, we turn an $O(N)$ query into $O(1)$.

```
class NumArray {
    long[] prefix;
    public NumArray(int[] nums) {
        prefix = new long[nums.length + 1];
        for (int i = 0; i < nums.length; i++) {
            prefix[i + 1] = prefix[i] + nums[i];
        }
    }
    public int sumRange(int i, int j) {
        return (int)(prefix[j + 1] - prefix[i]);
    }
}
```

2. Hashing the Prefix (Subarray Sum Equals K)

Theory: If $PrefixSum[j] - PrefixSum[i] = K$, then the sum of the subarray between i and j is K . We use a HashMap to store the frequency of prefix sums seen so far.

Why not Sliding Window? If the array has negative numbers, a window cannot be monotonic.

Hashing is the only $O(N)$ solution.

Question: Subarray Sum Equals K.

Logic: At each index, calculate current sum. Check if $sum - k$ exists in the map.

```

public int subarraySum(int[] nums, int k) {
    int count = 0, sum = 0;
    Map<Integer, Integer> map = new HashMap<>();
    map.put(0, 1); // Base case: A sum of 0 has appeared once

    for (int n : nums) {
        sum += n;
        if (map.containsKey(sum - k)) {
            count += map.get(sum - k);
        }
        map.put(sum, map.getOrDefault(sum, 0) + 1);
    }
    return count;
}

```

3. State Transformation (Equal 0s and 1s)

Theory: When asked for "Equal counts" of two things, transform the values into 1 and -1. The problem then becomes "Find a subarray that sums to 0."

Common variant: Longest subarray with equal vowels and consonants.

Question: Contiguous Array (Max length of subarray with equal 0s and 1s).

Logic: Replace 0 with -1. Use a HashMap to store the first index where a sum occurred to maximize length.

```

public int findMaxLength(int[] nums) {
    Map<Integer, Integer> map = new HashMap<>();
    map.put(0, -1); // Sum 0 at index -1
    int maxLen = 0, sum = 0;
    for (int i = 0; i < nums.length; i++) {
        sum += (nums[i] == 1 ? 1 : -1);
        if (map.containsKey(sum)) {
            maxLen = Math.max(maxLen, i - map.get(sum));
        } else {
            map.put(sum, i);
        }
    }
    return maxLen;
}

```

4. 2D Prefix Sum (Matrix Region Sum)

Theory: To find the sum of any rectangle in a matrix in $O(1)$, we pre-calculate sums from the origin (0,0).

Formula: $\text{Sum} = P[r2][c2] - P[r1-1][c2] - P[r2][c1-1] + P[r1-1][c1-1]$

Visual: You subtract the top and left rectangles, then add back the top-left corner because it was subtracted twice.

```
class NumMatrix {
    int[][] p;
    public NumMatrix(int[][] matrix) {
        int R = matrix.length, C = matrix[0].length;
        p = new int[R + 1][C + 1];
        for (int i = 0; i < R; i++)
            for (int j = 0; j < C; j++)
                p[i+1][j+1] = matrix[i][j] + p[i][j+1] + p[i+1][j] - p[i][j];
    }
    public int sumRegion(int r1, int c1, int r2, int c2) {
        return p[r2+1][c2+1] - p[r1][c2+1] - p[r2+1][c1] + p[r1][c1];
    }
}
```

5. Difference Array (Bulk Range Updates)

Theory: This is "Prefix Sum in reverse." If you need to add X to `nums[L...R]` many times, don't update N elements. Update two points in a diff array.

The Move: `diff[L] += X`, `diff[R+1] -= X`.

The Result: After all updates, the final array is the prefix sum of diff.

Question: Corporate Flight Bookings or Range Addition.

```

public int[] getModifiedArray(int length, int[][] updates) {
    int[] diff = new int[length];
    for (int[] u : updates) {
        diff[u[0]] += u[2];
        if (u[1] + 1 < length) diff[u[1] + 1] -= u[2];
    }
    for (int i = 1; i < length; i++) {
        diff[i] += diff[i - 1];
    }
    return diff;
}

```

Question Keyword	Logic Type	Key Data Structure
"Sum of range [i, j]"	Basic Prefix	long []
"Subarray sum equals K"	Freq Hashing	HashMap<Sum, Freq>
"Longest subarray sum K"	Index Hashing	HashMap<Sum, FirstIndex>
"Equal 0s and 1s"	Transformation	1 and -1 mapping
"Multiple range updates"	Difference Array	diff[L]+=v, diff[R+1]-=v
"Matrix / Rectangle Sum"	2D Prefix	int [R+1] [C+1]

Final Revision Note: Prefix sum is the "History" of the array. Whenever you need to know "Have I seen a state before that, when subtracted from my current state, gives me my target?", use Prefix Sum + HashMap.

Two Pointers

Two pointers typically fall into two categories: Opposite Ends (converging) and Slow-Fast (tracking/detecting).

1. Opposite Ends (The Search Space Reducer)

Theory: Start one pointer at 0 and one at n-1. Move them toward each other based on a condition.

Crucial Requirement: The input must usually be sorted.

HFT Tip: This is used in order-matching engines to find price pairs that sum to a target.

Question: Two Sum II - Input Array Is Sorted.

Logic: If $\text{sum} > \text{target}$, the only way to decrease the sum is to move the right pointer in. If $\text{sum} < \text{target}$, move left in.

```
public int[] twoSum(int[] numbers, int target) {
    int left = 0, right = numbers.length - 1;
    while (left < right) {
        int sum = numbers[left] + numbers[right];
        if (sum == target) return new int[]{left + 1, right + 1};
        if (sum < target) left++;
        else right--;
    }
    return new int[]{-1, -1};
}
```

2. Slow-Fast Pointers (The Tortoise and Hare)

Theory: One pointer moves at speed x and the other at $2x$.

Pattern A (Cycle Detection): If they meet, there is a cycle.

Pattern B (Middle Element): When the fast pointer reaches the end, the slow pointer is at the middle.

Question: Linked List Cycle.

Logic: In a circular path, the faster runner will eventually lap the slower runner.

```
public boolean hasCycle(ListNode head) {
    ListNode slow = head, fast = head;
    while (fast != null && fast.next != null) {
        slow = slow.next;
        fast = fast.next.next;
        if (slow == fast) return true;
    }
    return false;
}
```

3. Slow-Fast (In-place Modification)

Theory: Use one pointer to iterate (fast) and another to keep track of the "write" position (slow).

Why: FAANG loves this for "In-place" constraints (Space Complexity $O(1)$).

Question: Remove Duplicates from Sorted Array.

Logic: Only "write" to the slow index when the fast pointer finds a new unique element.

```
public int removeDuplicates(int[] nums) {
    if (nums.length == 0) return 0;
    int slow = 0;
    for (int fast = 1; fast < nums.length; fast++) {
        if (nums[fast] != nums[slow]) {
            slow++;
            nums[slow] = nums[fast];
        }
    }
    return slow + 1;
}
```

4. The "Three-Way" Partition (Dutch National Flag)

Theory: Use three pointers to sort an array containing three distinct values in $O(N)$ with one pass.

Pointers: low (boundary for 0s), mid (iterator/1s), high (boundary for 2s).

Question: Sort Colors (Sort 0s, 1s, and 2s).

Logic: Swap elements to their respective ends and shrink the boundaries.

```
public void sortColors(int[] nums) {
    int low = 0, mid = 0, high = nums.length - 1;
    while (mid <= high) {
        if (nums[mid] == 0) {
            swap(nums, low++, mid++);
        } else if (nums[mid] == 1) {
            mid++;
        } else {
            swap(nums, mid, high--); // Don't increment mid here, check swapped element
        }
    }
}
```

5. Two Arrays / Two Strings (The Merger)

Theory: One pointer for each data structure. Compare elements and move the pointer of the "chosen" element.

Question: Merge Sorted Array.

Logic: To do it in-place without extra space, start from the end of both arrays to avoid overwriting elements.

```
public void merge(int[] nums1, int m, int[] nums2, int n) {
    int i = m - 1, j = n - 1, k = m + n - 1;
    while (j >= 0) {
        if (i >= 0 && nums1[i] > nums2[j]) {
            nums1[k--] = nums1[i--];
        } else {
            nums1[k--] = nums2[j--];
        }
    }
}
```

Greedy

In Greedy problems, the "local optimum" usually involves sorting or using a Priority Queue to always pick the "best" current candidate.

1. Interval Scheduling (The Sorting Variant)

Theory: When dealing with time intervals, the greedy choice usually depends on sorting by Start Time or End Time.

Key Insight: To fit the most meetings, always pick the one that ends the earliest. This leaves the most room for future meetings.

Question: Non-overlapping Intervals (Min removals to make intervals non-overlapping).

Logic: Sort by end time. If the next interval starts before the current one ends, it must be removed.

```

public int eraseOverlapIntervals(int[][] intervals) {
    if (intervals.length == 0) return 0;
    // Sort by END time
    Arrays.sort(intervals, (a, b) -> Integer.compare(a[1], b[1]));

    int count = 0;
    int end = intervals[0][1];

    for (int i = 1; i < intervals.length; i++) {
        if (intervals[i][0] < end) {
            count++; // Overlap detected, "remove" this one
        } else {
            end = intervals[i][1]; // Update end to the current interval
        }
    }
    return count;
}

```

2. Jump Game (The "Reachable" Variant)

Theory: Instead of calculating every path (DP), track the furthest reachable index at any point.

Why Greedy? If you can reach index i , you can reach everything before i .

Question: Jump Game II (Minimum jumps to reach the end).

Logic: Maintain the currentMax reachable with N jumps and nextMax reachable with $N+1$ jumps.

```

public int jump(int[] nums) {
    int jumps = 0, currentEnd = 0, furthest = 0;
    for (int i = 0; i < nums.length - 1; i++) {
        furthest = Math.max(furthest, i + nums[i]);
        if (i == currentEnd) { // Must jump now
            jumps++;
            currentEnd = furthest;
        }
    }
    return jumps;
}

```

3. Two-Pass / Multi-Directional Greedy

Theory: Sometimes a local choice depends on both the left and right neighbors. Making one pass from left-to-right and another from right-to-left satisfies both constraints.

Question: Candy (Hard).

Logic: 1. Give everyone 1 candy.

2. Left-to-right: If $\text{rating}[i] > \text{rating}[i-1]$, $\text{candy}[i] = \text{candy}[i-1] + 1$.

3. Right-to-left: If $\text{rating}[i] > \text{rating}[i+1]$, $\text{candy}[i] = \max(\text{candy}[i], \text{candy}[i+1] + 1)$.

```
public int candy(int[] ratings) {
    int n = ratings.length;
    int[] candies = new int[n];
    Arrays.fill(candies, 1);

    for (int i = 1; i < n; i++)
        if (ratings[i] > ratings[i-1]) candies[i] = candies[i-1] + 1;

    for (int i = n - 2; i >= 0; i--)
        if (ratings[i] > ratings[i+1]) candies[i] = Math.max(candies[i], candies[i+1] + 1);

    return Arrays.stream(candies).sum();
}
```

4. Huffman/Priority Queue (The "Merge" Variant)

Theory: When you need to repeatedly pick the two smallest/largest elements to combine them.

This is common in HFT for building order books or calculating weighted costs.

Question: Minimum Cost to Connect Sticks.

Logic: Always pick the two shortest sticks, connect them, and put the result back into the pool.

```
public int connectSticks(int[] sticks) {
    PriorityQueue<Integer> pq = new PriorityQueue<>();
    for (int s : sticks) pq.add(s);

    int totalCost = 0;
    while (pq.size() > 1) {
        int cost = pq.poll() + pq.poll();
        totalCost += cost;
        pq.add(cost);
    }
    return totalCost;
}
```

5. Fractional Knapsack / Profit (HFT Style)

Theory: In HFT, you often deal with "Gas Station" or "Stock" problems where you have a limited

capacity but want to maximize return.

Logic: Calculate "Profit per unit of weight" and pick the highest.

Question: Gas Station.

Logic: If the total gas is less than total cost, it's impossible. Otherwise, if you run out of gas at station i , start over at $i+1$.

Question Keyword	Logic Type	Key Strategy
"Intervals", "Meetings"	Scheduling	Sort by End Time
"Can you reach..."	Reachability	Track <code>max_reachable</code>
"Minimize/Maximize total cost"	Huffman Style	PriorityQueue (Min-Heap)
"Left & Right neighbors"	Multi-Directional	Two passes ($L \rightarrow R$, $R \rightarrow L$)
"Smallest number of..."	Exchange Argument	Sort and pick largest/smallest

Greedy vs. DP: How to tell?

In a FAANG interview, if you are unsure:

Can I make a choice now that I won't regret later? (Greedy)

Does my current choice depend on future choices? (DP)

If a problem has "Optimal Substructure" and "Overlapping Subproblems," it's DP. If it has the Greedy Choice Property (a local optimum is part of a global optimum), it's Greedy.

HFT Pro-Tip: Greedy algorithms are preferred over DP in low-latency systems because they are $O(N)$ or $O(N \log N)$ and require significantly less memory (no DP table).

Recursion and BT

1. The "Pick / Not Pick" (Subsets Pattern)

Theory: At every single element, you have a binary choice: "Do I include this in my current set or not?" This generates 2

N

possibilities.

Key Use Case: Subsets, Power Sets, and problems where order doesn't matter but inclusion does.

Question: Subsets (Given an integer array, return all possible subsets).

```
public List<List<Integer>> subsets(int[] nums) {
    List<List<Integer>> res = new ArrayList<>();
    backtrack(0, nums, new ArrayList<>(), res);
    return res;
}

private void backtrack(int index, int[] nums, List<Integer> path, List<List<Integer>> r
    if (index == nums.length) {
        res.add(new ArrayList<>(path)); // Found a leaf node (a subset)
        return;
    }
    // Choice 1: Pick the element
    path.add(nums[index]);
    backtrack(index + 1, nums, path, res);

    // Choice 2: Not pick (Backtrack)
    path.remove(path.size() - 1);
    backtrack(index + 1, nums, path, res);
}
```

2. The "For-Loop" (Permutations Pattern)

Theory: At each level of recursion, you iterate through all available candidates. This is used when the order matters.

Key Use Case: Permutations, Combinations, and String arrangements.

Backtracking Logic: Swap or use a boolean[] used array to keep track of what's already in the "path."

Question: Permutations (Given an array, return all possible permutations).

```

private void backtrack(int[] nums, List<Integer> path, List<List<Integer>> res, boolean
    if (path.size() == nums.length) {
        res.add(new ArrayList<>(path));
        return;
    }
    for (int i = 0; i < nums.length; i++) {
        if (used[i]) continue; // Skip if already used in current path

        used[i] = true;
        path.add(nums[i]);
        backtrack(nums, path, res, used);

        // The "Backtrack" step
        path.remove(path.size() - 1);
        used[i] = false;
    }
}

```

3. The "State Exploration" (Constraint-Based)

Theory: This is for problems like Sudoku or N-Queens where you place an item, check if it's valid (constraints), and if it leads to a dead end, you remove it.

HFT Tip: Performance in backtracking comes from Pruning. If you can detect a failure early, you save thousands of recursive calls.

Question: N-Queens.

Logic: Place a queen in a row, check columns and diagonals. If valid, move to the next row. If no valid spot exists, go back and move the previous queen.

```

private void solve(int row, boolean[] cols, boolean[] d1, boolean[] d2, char[][] board)
    if (row == board.length) {
        // Add valid board to result
        return;
    }
    for (int col = 0; col < board.length; col++) {
        int id1 = row - col + board.length;
        int id2 = row + col;
        if (!cols[col] && !d1[id1] && !d2[id2]) {
            board[row][col] = 'Q';
            cols[col] = d1[id1] = d2[id2] = true; // Set state
            solve(row + 1, cols, d1, d2, board);
            board[row][col] = '.';
            cols[col] = d1[id1] = d2[id2] = false; // Undo state (Backtrack)
        }
    }
}

```

4. The "Path Finding" (Flood Fill / Matrix)

Theory: Exploring a 2D grid. You move Up, Down, Left, Right and mark cells as "visited" so you don't get stuck in an infinite loop.

Question: Word Search (Find if a word exists in a character grid).

Logic: For each cell, if the character matches, explore its neighbors. Unmark the cell after the neighbors are explored so it can be used in other paths.

```

public boolean backtrack(char[][] board, String word, int i, int j, int k) {
    if (k == word.length()) return true;
    if (i < 0 || j < 0 || i >= board.length || j >= board[0].length || board[i][j] != w

    char temp = board[i][j];
    board[i][j] = '#'; // Mark as visited

    boolean found = backtrack(board, word, i+1, j, k+1) ||
                    backtrack(board, word, i-1, j, k+1) ||
                    backtrack(board, word, i, j+1, k+1) ||
                    backtrack(board, word, i, j-1, k+1);

    board[i][j] = temp; // Unmark (Backtrack)
    return found;
}

```

Keyword in Question	Pattern	DS / Strategy	Complexity
"All Subsets"	Pick / Not Pick	index + 1 recursion	$O(2^N)$
"All Permutations"	For-Loop	boolean[] used array	$O(N!)$
"Find All Paths"	Grid Backtracking	visited[r][c] or In-place mark	$O(3^K)$ or $O(4^K)$
"Valid Sudoku/Queen"	Constraint BT	Check isValid() before call	High (but pruned)

The "Golden Rule" of Backtracking

If you find yourself stuck, remember the Three Pillars of Backtracking:

Our Choices: What decisions can we make at this step? (Iterate through nums, move 4 directions, etc.)

Our Constraints: What makes a choice "bad"? (Used already, out of bounds, board rules.)

Our Goal: When do we stop? (Target length reached, word found, end of array.)

HFT Pro-Tip: Backtracking is expensive. In production HFT code, we often replace recursion with a Stack-based iterative approach to avoid StackOverflowError and to better manage memory on the CPU cache.

Stack

1. Functional Stack (Nested Structures)

Theory: Used to handle LIFO (Last-In, First-Out) operations. It is the gold standard for parsing expressions, nested brackets, or "undo" operations.

HFT Tip: Use ArrayDeque instead of Stack. java.util.Stack is synchronized (thread-safe), which adds unnecessary overhead in single-threaded logic.

Question: Valid Parentheses.

Logic: Push the expected closing bracket onto the stack when an opening bracket is seen. This simplifies the comparison logic.

```

public boolean isValid(String s) {
    Deque<Character> stack = new ArrayDeque<>();
    for (char c : s.toCharArray()) {
        if (c == '(') stack.push('(');
        else if (c == '{') stack.push('{');
        else if (c == '[') stack.push('[');
        else if (stack.isEmpty() || stack.pop() != c) return false;
    }
    return stack.isEmpty();
}

```

2. Monotonic Stack (Next Greater Element)

Theory: Maintains elements in a specific order (increasing or decreasing). When a new element violates this order, we "process" the elements on the stack.

Key Insight: Storing indices is more powerful than storing values, as indices allow us to calculate the distance (time/width) between elements.

Question: Daily Temperatures.

Logic: Maintain a decreasing stack. If today's temperature is warmer than the temperature at the top index, pop and calculate the difference in days.

```

public int[] dailyTemperatures(int[] temperatures) {
    int n = temperatures.length;
    int[] res = new int[n];
    Deque<Integer> stack = new ArrayDeque<>(); // Store indices

    for (int i = 0; i < n; i++) {
        // While current temp is warmer than the stack top
        while (!stack.isEmpty() && temperatures[i] > temperatures[stack.peek()]) {
            int prevIndex = stack.pop();
            res[prevIndex] = i - prevIndex;
        }
        stack.push(i);
    }
    return res;
}

```

3. The "Area" Pattern (Largest Rectangle)

Theory: This is the most complex stack variant. To find the largest rectangle, for every bar *i*, we need the first smaller bar to its left and the first smaller bar to its right.

The Trick: We use a Monotonic Increasing stack. When we see a height smaller than the top, the top element is the "height," the current index is the "right boundary," and the element below the top is the "left boundary."

Question: Largest Rectangle in Histogram.

```
public int largestRectangleArea(int[] heights) {
    Deque<Integer> stack = new ArrayDeque<>();
    stack.push(-1); // Sentinel value to handle the left boundary
    int maxArea = 0;
    int n = heights.length;

    for (int i = 0; i < n; i++) {
        // Maintain monotonic increasing order
        while (stack.peek() != -1 && heights[stack.peek()] >= heights[i]) {
            int currentHeight = heights[stack.pop()];
            // Width is current index - new top index - 1
            int width = i - stack.peek() - 1;
            maxArea = Math.max(maxArea, currentHeight * width);
        }
        stack.push(i);
    }

    // Cleanup: Process remaining elements in the stack
    // These bars have no smaller element to their right
    while (stack.peek() != -1) {
        int currentHeight = heights[stack.pop()];
        int width = n - stack.peek() - 1;
        maxArea = Math.max(maxArea, currentHeight * width);
    }

    return maxArea;
}
```

4. The "Trapping Rain Water" (Stack Variant)

Theory: While often solved with Two Pointers, the Stack approach is very common in FAANG. It processes the water "horizontally" (row by row).

Logic: When we find a bar taller than the top of the stack, it forms a container. The popped element is the bottom of the container, and the new top and current index are the walls.

```

public int trap(int[] height) {
    int water = 0;
    Deque<Integer> stack = new ArrayDeque<>();
    for (int i = 0; i < height.length; i++) {
        while (!stack.isEmpty() && height[i] > height[stack.peek()]) {
            int bottomIndex = stack.pop();
            if (stack.isEmpty()) break; // No left wall

            int leftWallIndex = stack.peek();
            int h = Math.min(height[i], height[leftWallIndex]) - height[bottomIndex];
            int w = i - leftWallIndex - 1;
            water += h * w;
        }
        stack.push(i);
    }
    return water;
}

```

HFT & FAANG Revision Tips

Why indices? Storing indices allows you to compute width or time_difference as `current_index - stack.peek() - 1`.

Sentinel Elements: Adding a 0 to the end of your input array (for Histograms) or a -1 to the stack helps avoid the "cleanup" while loop and keeps the code dry.

Space Complexity: Stacks are $O(N)$ space. In HFT, if you know the maximum nesting depth or the maximum "span" of a monotonic lookup, you can pre-allocate a primitive array to act as a stack for zero-allocation performance.

Queue

1. Standard Queue (FIFO - First-In, First-Out)

Theory: Elements are added at the back and removed from the front.

Practical Use Cases:

BFS (Breadth-First Search): Level-order traversal of trees/graphs.

Task Scheduling: Managing requests in a web server or print jobs.

HFT Message Buffers: Handling incoming market data ticks in the order they arrive.

Question: Binary Tree Level Order Traversal.

Logic: Use a queue to keep track of nodes at the current level. Before moving to the next level, process all nodes currently in the queue.

```
public List<List<Integer>> levelOrder(TreeNode root) {
    List<List<Integer>> result = new ArrayList<>();
    if (root == null) return result;

    Queue<TreeNode> queue = new LinkedList<>();
    queue.offer(root);

    while (!queue.isEmpty()) {
        int levelSize = queue.size(); // Lock the size for the current level
        List<Integer> currentLevel = new ArrayList<>();

        for (int i = 0; i < levelSize; i++) {
            TreeNode node = queue.poll();
            currentLevel.add(node.val);
            if (node.left != null) queue.offer(node.left);
            if (node.right != null) queue.offer(node.right);
        }
        result.add(currentLevel);
    }
    return result;
}
```

2. Monotonic Queue (Sliding Window Extremes)

Theory: A specialized Deque (Double-Ended Queue) that maintains elements in a strictly increasing or decreasing order.

Practical Use Cases:

Sliding Statistics: Finding the maximum or minimum price in a sliding window of a stock's 1-minute interval.

Resource Allocation: Managing a buffer where only the "best" (max/min) candidates are relevant.

Question: Sliding Window Maximum.

Logic: Maintain a decreasing deque of indices.

If the front index is out of the window range, remove it.

If the current element is larger than the elements at the back of the deque, remove those back elements (they can never be the max again).

The element at the front is always the maximum for the current window.

```
public int[] maxSlidingWindow(int[] nums, int k) {
    if (nums == null || nums.length == 0) return new int[0];
    int n = nums.length;
    int[] res = new int[n - k + 1];
    Deque<Integer> dq = new ArrayDeque<>(); // Stores indices

    for (int i = 0; i < n; i++) {
        // 1. Remove indices that are out of the current window
        if (!dq.isEmpty() && dq.peekFirst() == i - k) {
            dq.pollFirst();
        }

        // 2. Remove smaller elements from the back (Monotonic Property)
        // They are useless because nums[i] is larger and lasts longer
        while (!dq.isEmpty() && nums[dq.peekLast()] < nums[i]) {
            dq.pollLast();
        }

        dq.offerLast(i);

        // 3. The first element in the deque is the max for the current window
        if (i >= k - 1) {
            res[i - k + 1] = nums[dq.peekFirst()];
        }
    }
    return res;
}
```

3. The "Constrained Subsequence" Pattern (Hard)

Theory: Combining Prefix Sums with a Monotonic Queue. This is common in HFT for optimizing "Shortest/Longest" path constraints over a rolling time window.

Question: Shortest Subarray with Sum at Least K.

Logic: Use a monotonic increasing deque of prefix sums.

We want to find $\text{Prefix}[i] - \text{Prefix}[j] \geq K$ where $i - j$ is minimized.

The deque helps us efficiently find the best j for a given i.

```
public int shortestSubarray(int[] nums, int k) {
    int n = nums.length;
    long[] prefixSums = new long[n + 1];
    for (int i = 0; i < n; i++) prefixSums[i + 1] = prefixSums[i] + nums[i];

    int minLen = n + 1;
    Deque<Integer> dq = new ArrayDeque<>();

    for (int i = 0; i <= n; i++) {
        // Find valid subarray and try to shrink it
        while (!dq.isEmpty() && prefixSums[i] - prefixSums[dq.peekFirst()] >= k) {
            minLen = Math.min(minLen, i - dq.pollFirst());
        }
        // Maintain monotonic increasing prefix sums
        while (!dq.isEmpty() && prefixSums[i] <= prefixSums[dq.peekLast()]) {
            dq.pollLast();
        }
        dq.offerLast(i);
    }
    return minLen <= n ? minLen : -1;
}
```

Scenario	Logic Type	Key Strategy
"Level by level" traversal	Standard Queue	queue.size() loop inside while
"Max/Min in sliding window"	Monotonic Queue	dq.pollLast() if nums[i] > back
"Shortest subarray sum $\geq K$ "	Monotonic + Prefix	Increasing deque of prefix indices
"First unique char in stream"	Queue + Hash	Remove from front if non-unique

Practical & HFT Tips

Deque for the win: In Java, ArrayDeque is your best friend. It can act as a Stack, a Queue, and a Monotonic Queue. It is faster than LinkedList and Stack.

Order Matters: In HFT, queues represent the Order Book. Monotonic properties are used to find the "Best Bid" or "Best Offer" efficiently as new orders arrive and old ones expire.

Indices vs. Values: Just like Monotonic Stacks, always store indices. The index tells you both the value (nums[i]) and the position (to check if it's still in the window).

Binary Search

In HFT and high-level interviews, Binary Search is the go-to for continuous functions and optimization problems.

The Binary Search Compendium

1. The Classic Search (Iterative Template)

Theory: Used for exact matches in a sorted range.

FAANG Tip: Always use $\text{mid} = \text{left} + (\text{right} - \text{left}) / 2$ to avoid integer overflow.

Question: Search in Rotated Sorted Array.

Logic: Even when rotated, one half of the array is always sorted. Use that sorted half to decide where to move.

```
public int search(int[] nums, int target) {
    int left = 0, right = nums.length - 1;
    while (left <= right) {
        int mid = left + (right - left) / 2;
        if (nums[mid] == target) return mid;

        // Left half is sorted
        if (nums[left] <= nums[mid]) {
            if (target >= nums[left] && target < nums[mid]) right = mid - 1;
            else left = mid + 1;
        }
        // Right half is sorted
        else {
            if (target > nums[mid] && target <= nums[right]) left = mid + 1;
            else right = mid - 1;
        }
    }
    return -1;
}
```

2. Binary Search on Answer (The "Max-Min" Pattern)

Theory: This is the most common Medium/Hard variant. Instead of searching an array, you search a range of possible values for the answer.

Clue: The question asks for the "Minimum maximum" or "Maximum minimum."

Requirement: A check(value) function that is monotonic (if X works, X+1 also works).

Question: Koko Eating Bananas.

Logic: The possible speeds are between 1 and max(bananas). Binary search for the smallest speed v that allows Koko to finish within H hours.

```
public int minEatingSpeed(int[] piles, int h) {
    int left = 1, right = 1_000_000_000; // Search space for "speed"
    while (left < right) {
        int mid = left + (right - left) / 2;
        if (canFinish(piles, mid, h)) right = mid;
        else left = mid + 1;
    }
    return left;
}

private boolean canFinish(int[] piles, int speed, int h) {
    long hours = 0;
    for (int p : piles) {
        hours += (p + speed - 1) / speed; // Ceiling division
    }
    return hours <= h;
}
```

3. Boundary Search (Lower/Upper Bound)

Theory: Finding the first or last occurrence of an element, or the first element that satisfies a condition.

Trick: left < right with right = mid or left = mid + 1. This avoids infinite loops and correctly lands on the boundary.

Question: First and Last Position of Element.

```
private int findBound(int[] nums, int target, boolean isFirst) {
    int left = 0, right = nums.length - 1, res = -1;
    while (left <= right) {
        int mid = left + (right - left) / 2;
        if (nums[mid] == target) {
            res = mid;
            if (isFirst) right = mid - 1; // Keep looking left
            else left = mid + 1;          // Keep looking right
        } else if (nums[mid] < target) left = mid + 1;
        else right = mid - 1;
    }
    return res;
}
```

4. Binary Search on Matrix (2D)

Theory: Treat a 2D matrix as a virtual 1D array.

Mapping: For a matrix of size $M \times N$, index i maps to $\text{matrix}[i / N][i \% N]$.

5. Peak Finding (Binary Search on Unsorted)

Theory: You can use Binary Search on unsorted data if there is a local property you can follow.

Logic: If $\text{nums}[\text{mid}] < \text{nums}[\text{mid} + 1]$, there must be a peak to the right.

Question: Find Peak Element.

```
public int findPeakElement(int[] nums) {
    int left = 0, right = nums.length - 1;
    while (left < right) {
        int mid = left + (right - left) / 2;
        if (nums[mid] < nums[mid + 1]) left = mid + 1;
        else right = mid;
    }
    return left;
}
```

Question Keyword	Pattern	Strategy
"Sorted" + "Find index"	Classic BS	while (l <= r)
"Min of Max" / "Max of Min"	BS on Answer	check(mid) function

Question Keyword	Pattern	Strategy
"First occurrence"	Boundary Search	<code>right = mid</code> or <code>left = mid + 1</code>
"Rotated sorted"	Modified BS	Find the sorted half first
"Peak" / "Local property"	Peak Search	Compare <code>mid</code> with <code>mid + 1</code>

1. Search Space: In "BS on Answer," the hardest part is defining the left and right boundaries. Usually, `left` = min possible answer and `right` = max possible answer.
2. Monotonicity: Before using BS, ask: "If I increase the input, does the result change in only one direction?" If yes, BS is applicable.
3. HFT Context: In low-latency trading, Binary Search is used in Order Matching (finding the best price in a sorted Order Book) and Risk Management (finding the threshold at which a trade violates a constraint).
4. Floating Point BS: If searching for a decimal (e.g., `sqrt(x)`), use a fixed number of iterations (e.g., `for(int i=0; i<100; i++)`) to avoid precision issues with `while(left < right)`.

Divide and Conquer

Divide and Conquer (D&C) is a high-level algorithm design paradigm. It isn't a single "trick" like a sliding window; it's a philosophy of problem-solving that powers some of the fastest algorithms in the world (like Quicksort and the Fast Fourier Transform used in HFT).

The Three-Step Philosophy

Divide: Break the big problem into smaller, identical sub-problems.

Conquer: Solve the sub-problems recursively. (If they are small enough, solve them directly as "base cases").

Combine: Stitch the answers of the sub-problems together to solve the original big problem.

✂ The Divide & Conquer Compendium

1. Merge Sort (The "Combine" Masterclass)

Theory: To sort an array, split it in half, sort each half, and then "merge" the two sorted halves.

The "Divide" is easy, but the "Combine" (Merge) is where the work happens.

Why use it? It's a stable $O(N \log N)$ sort. In HFT, Merge Sort is used when the relative order of equal-priced trades must be preserved.

```
public void mergeSort(int[] arr, int l, int r) {
    if (l < r) {
        int m = l + (r - l) / 2;
        mergeSort(arr, l, m);    // Divide & Conquer Left
        mergeSort(arr, m + 1, r); // Divide & Conquer Right
        merge(arr, l, m, r);     // Combine
    }
}

private void merge(int[] arr, int l, int m, int r) {
    // Standard logic to merge two sorted arrays into one
}
```

2. Quick Select (The "Partition" Variant)

Theory: Based on Quicksort logic. Instead of sorting the whole array, you "partition" the array around a pivot. If the pivot lands on the index you want (like the Median), you stop.

Why use it? It finds the k th largest element in $O(N)$ average time, much faster than sorting the whole array ($O(N \log N)$).

Question: Kth Largest Element in an Array.

```
public int findKthLargest(int[] nums, int k) {
    return quickSelect(nums, 0, nums.length - 1, nums.length - k);
}

private int quickSelect(int[] nums, int left, int right, int k) {
    int pivotIndex = partition(nums, left, right);
    if (pivotIndex == k) return nums[k];
    return pivotIndex < k
        ? quickSelect(nums, pivotIndex + 1, right, k)
        : quickSelect(nums, left, pivotIndex - 1, k);
}
```

3. Advanced Binary Trees (Divide & Conquer on Structures)

Theory: Almost all Binary Tree problems are D&C. You solve the left subtree, solve the right subtree, and combine the results at the root.

Question: Construct Binary Tree from Preorder and Inorder Traversal.

Logic: The first element of Preorder is the Root. Find this root in Inorder to Divide the array into Left and Right subtrees. Repeat.

```
public TreeNode buildTree(int[] preorder, int[] inorder) {
    return helper(0, 0, inorder.length - 1, preorder, inorder);
}

private TreeNode helper(int preStart, int inStart, int inEnd, int[] preorder, int[] ino
    if (preStart > preorder.length - 1 || inStart > inEnd) return null;

    TreeNode root = new TreeNode(preorder[preStart]);
    int inIndex = 0; // Find index of root in inorder array
    for (int i = inStart; i <= inEnd; i++) {
        if (inorder[i] == root.val) inIndex = i;
    }

    // Divide and Conquer
    root.left = helper(preStart + 1, inStart, inIndex - 1, preorder, inorder);
    root.right = helper(preStart + inIndex - inStart + 1, inIndex + 1, inEnd, preorder,

    return root;
}
```

4. Closest Pair of Points (Geometry + D&C)

Theory: Finding the two closest points in a 2D plane. Brute force is $O(N^2)$. D&C makes it $O(N \log N)$.

Divide: Split points by X-coordinate.

Conquer: Find the min distance δ in the left and right halves.

Combine: Check points near the "split line" that might be closer than δ .

Question Keyword	Pattern	Why Divide & Conquer?
"Sort this..."	Merge/Quick Sort	Reduces $O(N^2)$ logic to $O(N \log N)$.
"Kth largest/smallest"	QuickSelect	Prunes half the array each step.
"Tree Traversal"	Tree D&C	Subtrees are independent sub-problems.

Question Keyword	Pattern	Why Divide & Conquer?
"Skyline Problem"	Merge Style	Merging two "skylines" into one.
"Matrix Multiplication"	Strassen's	Optimized sub-matrix operations.

Why use D&C? (The FAANG Perspective)

Parallelization: Because sub-problems are independent, you can solve the left half on CPU Core 1 and the right half on CPU Core 2. This is huge in HFT.

Memory Efficiency: It often uses the Call Stack to manage data, which can be more cache-friendly than massive DP tables.

Logarithmic Complexity: Whenever you see a problem where you can ignore half the data or combine two halves efficiently, D&C will likely give you that coveted $\log N$ factor.

Comparison Tip:

Greedy: Pick the best local move and never look back.

Backtracking: Try a move, and if it fails, go back and try another.

Divide & Conquer: Break the problem down, solve all pieces, and glue them back together.

Hashing

1. Rolling Hash (Rabin-Karp Algorithm)

Theory: Instead of re-hashing a sliding window of size L from scratch (which would take $O(L)$), you "roll" the hash in $O(1)$. You subtract the high-order digit, multiply by the base, and add the new digit.

HFT Tip: Use a large prime modulus (2

61

–1 is a favorite) to prevent "hash collisions" where two different strings produce the same hash.

Question: Repeated DNA Sequences (Find all 10-letter-long sequences that occur more than once).

```

public List<String> findRepeatedDnaSequences(String s) {
    if (s.length() < 10) return new ArrayList<>();
    Set<Integer> seen = new HashSet<>();
    Set<String> res = new HashSet<>();

    // Map characters to 2-bit values (A=0, C=1, G=2, T=3)
    int[] map = new int[256];
    map['A'] = 0; map['C'] = 1; map['G'] = 2; map['T'] = 3;

    int hash = 0;
    for (int i = 0; i < s.length(); i++) {
        hash = (hash << 2) | map[s.charAt(i)]; // Shift left 2 bits, add new char
        hash &= 0xFFFF; // Mask to keep only the last 20 bits (10 chars * 2 bits)

        if (i >= 9) {
            if (!seen.add(hash)) res.add(s.substring(i - 9, i + 1));
        }
    }
    return new ArrayList<>(res);
}

```

2. Structural Hashing (Merkle/Tree Hashing)

Theory: To check if two large trees are identical, you hash the nodes based on their value and the hashes of their children. If the root hashes match, the trees are identical.

Why: In FAANG, this is used in Git and Blockchain to verify data integrity without comparing every single byte.

Question: Find Duplicate Subtrees.

```

public List<TreeNode> findDuplicateSubtrees(TreeNode root) {
    List<TreeNode> res = new ArrayList<>();
    Map<String, Integer> map = new HashMap<>();
    traverse(root, map, res);
    return res;
}

private String traverse(TreeNode node, Map<String, Integer> map, List<TreeNode> res) {
    if (node == null) return "#";
    // Construct a unique hash-string for the current subtree
    String serial = node.val + "," + traverse(node.left, map, res) + "," + traverse(node.right, map, res);

    map.put(serial, map.getOrDefault(serial, 0) + 1);
    if (map.get(serial) == 2) res.add(node);
    return serial;
}

```

3. Spatial Hashing (Grid Hashing)

Theory: Instead of comparing all points $O(N^2)$

2

) to find pairs within a certain distance, you map points to a grid cell (bucket) based on coordinates.

Formula: $\text{key} = (\text{floor}(x/\text{gridSize}), \text{floor}(y/\text{gridSize}))$

HFT Use Case: Used in high-speed geometric matching and limit-order book clustering.

Question: Max Points on a Line.

Logic: For a fixed point i , calculate the slope to every other point j . Store slopes in a HashMap. A slope is best represented as a string or a custom Pair of simplified GCD values to avoid double precision issues.

```

public int maxPoints(int[][] points) {
    int n = points.length;
    if (n <= 2) return n;
    int max = 0;
    for (int i = 0; i < n; i++) {
        Map<String, Integer> map = new HashMap<>();
        int localMax = 0;
        for (int j = i + 1; j < n; j++) {
            int dx = points[j][0] - points[i][0];
            int dy = points[j][1] - points[i][1];
            int common = gcd(dx, dy); // Simplify fraction
            String slope = (dx / common) + "/" + (dy / common);
            map.put(slope, map.getOrDefault(slope, 0) + 1);
            localMax = Math.max(localMax, map.get(slope));
        }
        max = Math.max(max, localMax + 1);
    }
    return max;
}

private int gcd(int a, int b) {
    return b == 0 ? a : gcd(b, a % b);
}

```

4. Probabilistic Hashing (Bloom Filter Concept)

Theory: While typically a System Design topic, FAANG coding interviews sometimes ask for "Memory Efficient" membership checks. You use multiple hash functions on the same bitset.

Question: Design HashSet (With limited space).

5. Consistent Hashing (The Circle Pattern)

Theory: In distributed systems, hashing a key to a server using $\text{hash} \% N$ is brittle. Consistent hashing maps keys and servers to a unit circle (0 to 2

32

−1).

Logic: A key is assigned to the first server it encounters moving clockwise on the circle. Use a TreeMap in Java to simulate the circle.

```

class ConsistentHash {
    private final TreeMap<Integer, String> circle = new TreeMap<>();

    public void addServer(String server) {
        // In reality, add multiple virtual nodes for better distribution
        circle.put(server.hashCode(), server);
    }

    public String getServer(String key) {
        if (circle.isEmpty()) return null;
        int hash = key.hashCode();
        // Find the "ceiling" key in the TreeMap
        Integer serverHash = circle.ceilingKey(hash);
        // If we reach the end of the circle, wrap around to the first server
        if (serverHash == null) serverHash = circle.firstKey();
        return circle.get(serverHash);
    }
}

```

Technique	DS to Use	Identifying Clue
Rolling Hash	long / Polynomial	"Find substring of length L", "DNA sequences"
Structural Hash	String / Serialization	"Find duplicate subtrees", "Tree similarity"
Spatial Hashing	HashMap<String, List>	"Points within distance D", "Collision detection"
Slope Hashing	HashMap + GCD	"Points on the same line", "Geometry"
Consistent Hash	TreeMap	"Distribute keys across servers", "System Design"

HFT & FAANG Revision Tips

Avoid Floats: Never use Double as a HashMap key (e.g., for slopes). Due to precision errors, 0.333333333 may not equal 0.333333334. Use a simplified String like "1/3".

The Modulo Trick: In Rolling Hash, always add the mod before taking the final modulo to handle negative results: $(h + \text{mod}) \% \text{mod}$.

Pre-sizing: In Java, if you know you will store N elements, initialize the map with new HashMap<>((int) (N / 0.75) + 1) to prevent expensive rehashing during execution.

Arrays and Strings

1. Kadane's Algorithm (Subarray Sum Optimization)

Theory: Used to find the maximum sum of a contiguous subarray. At each index, you decide: "Is it better to start a new subarray here, or continue the existing one?"

Trick: If currentSum becomes negative, reset it to 0.

Question: Maximum Subarray.

```
public int maxSubArray(int[] nums) {  
    int maxSoFar = nums[0];  
    int currentMax = nums[0];  
    for (int i = 1; i < nums.length; i++) {  
        // Local choice: start fresh at i, or add i to currentMax  
        currentMax = Math.max(nums[i], currentMax + nums[i]);  
        maxSoFar = Math.max(maxSoFar, currentMax);  
    }  
    return maxSoFar;  
}
```

2. Boyer-Moore Voting (Majority Element)

Theory: A brilliant O(1) space algorithm to find an element that appears more than n/2 times.

Mechanism: You maintain a "candidate" and a "count." If the next element is the candidate, increment count; otherwise, decrement. When count hits zero, pick the next element as the new candidate.

Question: Majority Element.

```
public int majorityElement(int[] nums) {  
    int count = 0, candidate = 0;  
    for (int num : nums) {  
        if (count == 0) candidate = num;  
        count += (num == candidate) ? 1 : -1;  
    }  
    return candidate;  
}
```

3. String Manipulation: KMP (Knuth-Morris-Pratt)

Theory: Used for string matching in $O(N+M)$ instead of $O(N \cdot M)$. It uses a "LPS" (Longest Prefix that is also a Suffix) table to avoid backtracking in the main string.

HFT Tip: This is critical for matching "symbols" or "packets" in a continuous data stream.

Question: Find the Index of the First Occurrence in a String.

```

public int strStr(String haystack, String needle) {
    if (needle.isEmpty()) return 0;
    int[] lps = computeLPS(needle);
    int i = 0, j = 0;
    while (i < haystack.length()) {
        if (haystack.charAt(i) == needle.charAt(j)) {
            i++; j++;
            if (j == needle.length()) return i - j;
        } else if (j > 0) {
            j = lps[j - 1]; // Jump based on LPS
        } else {
            i++;
        }
    }
    return -1;
}

private int[] computeLPS(String pattern) {
    int[] lps = new int[pattern.length()];
    int len = 0, i = 1;
    while (i < pattern.length()) {
        if (pattern.charAt(i) == pattern.charAt(len)) {
            lps[i++] = ++len;
        } else if (len > 0) {
            len = lps[len - 1];
        } else {
            lps[i++] = 0;
        }
    }
    return lps;
}

```

4. Cyclic Sort (Range [1,N] Pattern)

Theory: If an array contains numbers from 1 to N, the number i should be at index i-1. You iterate and swap elements to their correct positions until the array is "sorted."

Clue: "Given an array of size N containing numbers from 1 to N..."

Question: First Missing Positive.

```

public int firstMissingPositive(int[] nums) {
    int n = nums.length;
    for (int i = 0; i < n; i++) {
        // While current number is in range and not in its correct spot
        while (nums[i] > 0 && nums[i] <= n && nums[nums[i] - 1] != nums[i]) {
            int temp = nums[nums[i] - 1];
            nums[nums[i] - 1] = nums[i];
            nums[i] = temp;
        }
    }
    for (int i = 0; i < n; i++) {
        if (nums[i] != i + 1) return i + 1;
    }
    return n + 1;
}

```

5. String Expansion (Palindromic Spread)

Theory: Instead of checking all $O(N^2)$

2

) substrings for palindromes, expand from each character (odd length) and between each character (even length).

Question: Longest Palindromic Substring.

```

public String longestPalindrome(String s) {
    int start = 0, end = 0;
    for (int i = 0; i < s.length(); i++) {
        int len1 = expand(s, i, i);    // Odd
        int len2 = expand(s, i, i + 1); // Even
        int len = Math.max(len1, len2);
        if (len > end - start) {
            start = i - (len - 1) / 2;
            end = i + len / 2;
        }
    }
    return s.substring(start, end + 1);
}

private int expand(String s, int left, int right) {
    while (left >= 0 && right < s.length() && s.charAt(left) == s.charAt(right)) {
        left--; right++;
    }
    return right - left - 1;
}

```

6. Matrix: Spiral & Transpose

Theory: 2D arrays are just arrays of arrays.

Transpose: $\text{matrix}[i][j] \leftrightarrow \text{matrix}[j][i]$ (Flip over diagonal).

Spiral: Maintain 4 boundaries (top, bottom, left, right) and shrink them as you traverse.

Question Clue	Pattern	Strategy
"Maximum contiguous sum"	Kadane's	<code>currentMax = max(n, curr+n)</code>
"Numbers from 1 to N"	Cyclic Sort	<code>swap(nums[i], nums[nums[i]-1])</code>
"Subsequence / Pattern"	KMP / Z-Algo	Precompute jump table
"Rotate Matrix"	Transpose + Reverse	Flip then reverse rows
"Majority element"	Boyer-Moore	Count +1 / -1 for candidate

HFT & FAANG Revision Tips

In-Place is King: In HFT, memory allocation is expensive. Practice converting String to char[] and modifying the array directly rather than creating new String objects.

ASCII vs Unicode: Clarify if "Strings" means ASCII (256 chars) or Unicode. If it's ASCII, an `int[256]` is always faster than a `HashMap`.

`StringBuilder`: In Java, never do `str += char` in a loop ($O(N^2)$). Always use `StringBuilder` ($O(N)$).

Edge Cases: Always check for null, length 0, and length 1. For integers, check for `Integer.MIN_VALUE` before negating (as `-Integer.MIN_VALUE` overflows).

Arrays and Strings 2

1. The Prefix/Suffix Product Pattern

Theory: To avoid $O(N^2)$

) or division, calculate the state of everything to the left of `i`, then everything to the right of `i`, and multiply them.

Question: Product of Array Except Self.

Logic: Use one pass to store the prefix product in the result array, then a second pass (going backward) to multiply by the suffix product.

```

public int[] productExceptSelf(int[] nums) {
    int n = nums.length;
    int[] res = new int[n];
    res[0] = 1;
    // Prefix product
    for (int i = 1; i < n; i++) {
        res[i] = res[i - 1] * nums[i - 1];
    }
    // Suffix product (running variable)
    int right = 1;
    for (int i = n - 1; i >= 0; i--) {
        res[i] *= right;
        right *= nums[i];
    }
    return res;
}

```

2. The "State Discovery" (Lexicographical) Pattern

Theory: Finding the "next" logical sequence requires identifying the first point of "decrease" from the end.

Question: Next Permutation.

Logic:

Find the first pair $\text{nums}[i] < \text{nums}[i+1]$ from the right.

Find the smallest number in the suffix $[i+1, n-1]$ that is larger than $\text{nums}[i]$.

Swap them.

Reverse the suffix $[i+1, n-1]$ to make it as small as possible.

```

public void nextPermutation(int[] nums) {
    int i = nums.length - 2;
    while (i >= 0 && nums[i] >= nums[i + 1]) i--; // Find pivot
    if (i >= 0) {
        int j = nums.length - 1;
        while (nums[j] <= nums[i]) j--; // Find successor
        swap(nums, i, j);
    }
    reverse(nums, i + 1); // Reverse suffix
}

```

3. Bucket Sort (O(N) Top-K Patterns)

Theory: If the range of values (or frequencies) is bounded by N (the array size), you can use the indices of an array as "buckets" to sort in linear time.

Question: Top K Frequent Elements.

Logic: 1. Map frequencies: Map<Integer, Integer>.

2. Create List[] bucket where bucket[i] stores elements with frequency i.

3. Iterate from the end of the bucket array to pick the top K.

```
public int[] topKFrequent(int[] nums, int k) {
    Map<Integer, Integer> map = new HashMap<>();
    for (int n : nums) map.put(n, map.getOrDefault(n, 0) + 1);

    List<Integer>[] bucket = new List[nums.length + 1];
    for (int key : map.keySet()) {
        int freq = map.get(key);
        if (bucket[freq] == null) bucket[freq] = new ArrayList<>();
        bucket[freq].add(key);
    }

    int[] res = new int[k];
    int counter = 0;
    for (int i = bucket.length - 1; i >= 0 && counter < k; i--) {
        if (bucket[i] != null) {
            for (int num : bucket[i]) {
                res[counter++] = num;
                if (counter == k) break;
            }
        }
    }
    return res;
}
```

4. Quickselect (Statistical Order Statistics)

Theory: Based on the Quicksort partition. It finds the kth element in O(N) average time.

Question: Kth Largest Element in an Array.

```

public int findKthLargest(int[] nums, int k) {
    return quickSelect(nums, 0, nums.length - 1, nums.length - k);
}

private int quickSelect(int[] nums, int left, int right, int k) {
    int pivot = nums[right], p = left;
    for (int i = left; i < right; i++) {
        if (nums[i] <= pivot) swap(nums, i, p++);
    }
    swap(nums, p, right);
    if (p == k) return nums[p];
    return p < k ? quickSelect(nums, p + 1, right, k) : quickSelect(nums, left, p - 1, k);
}

```

5. The "Sliding Window + Bucket" (Contains Duplicate III)

Theory: When you need to find $|nums[i] - nums[j]| \leq t$ within a distance k , you use "buckets" of size $t+1$. Any two numbers in the same bucket satisfy the condition.

Question: Contains Duplicate III.

```

public boolean containsNearbyAlmostDuplicate(int[] nums, int k, int t) {
    if (t < 0) return false;
    Map<Long, Long> buckets = new HashMap<>();
    long w = (long)t + 1;
    for (int i = 0; i < nums.length; i++) {
        long m = getID(nums[i], w);
        if (buckets.containsKey(m)) return true;
        if (buckets.containsKey(m - 1) && Math.abs(nums[i] - buckets.get(m - 1)) < w) r
        if (buckets.containsKey(m + 1) && Math.abs(nums[i] - buckets.get(m + 1)) < w) r
        buckets.put(m, (long)nums[i]);
        if (i >= k) buckets.remove(getID(nums[i - k], w));
    }
    return false;
}

private long getID(long i, long w) {
    return i < 0 ? (i + 1) / w - 1 : i / w;
}

```

1. Difference Array (Range Updates)

Theory: If you need to perform M updates (adding X to the range $[L, R]$) and then query the final values, don't update every element. Just update the boundaries.

Formula: $\text{diff}[L] += \text{val}$, $\text{diff}[R + 1] -= \text{val}$.

The "Magic": The prefix sum of the diff array gives you the final values after all updates.

Question: Range Addition / Corporate Flight Bookings.

```
public int[] getModifiedArray(int length, int[][] updates) {
    int[] diff = new int[length];
    for (int[] update : updates) {
        int start = update[0], end = update[1], val = update[2];
        diff[start] += val;
        if (end + 1 < length) diff[end + 1] -= val;
    }
    // Convert diff array to prefix sum array
    for (int i = 1; i < length; i++) {
        diff[i] += diff[i - 1];
    }
    return diff;
}
```

2. In-Place State Flagging (The "Flag" Pattern)

Theory: When an array contains numbers in a fixed range (like $[0, N]$), you can use the sign of the numbers at specific indices to store boolean information ("Have I seen this?").

Why: It allows for $O(1)$ extra space by reusing the input array as a hash set.

Question: Find All Numbers Disappeared in an Array.

Logic: For every number x you see, go to index $|x|-1$ and make the number there negative. Any index that remains positive at the end corresponds to a missing number.

```
public List<Integer> findDisappearedNumbers(int[] nums) {
    for (int i = 0; i < nums.length; i++) {
        int index = Math.abs(nums[i]) - 1;
        if (nums[index] > 0) nums[index] = -nums[index];
    }
    List<Integer> res = new ArrayList<>();
    for (int i = 0; i < nums.length; i++) {
        if (nums[i] > 0) res.add(i + 1);
    }
    return res;
}
```

3. Two-Pass Trapping (The "Water" Pattern)

Theory: Some problems require global knowledge from both sides. You calculate a "Max from Left" array and a "Max from Right" array. The answer at index i is usually a function of both.

Question: Trapping Rain Water.

Logic: Water at index i is trapped by $\min(\text{maxLeft}, \text{maxRight}) - \text{height}[i]$.

```
public int trap(int[] height) {
    int n = height.length;
    if (n == 0) return 0;
    int[] leftMax = new int[n], rightMax = new int[n];

    leftMax[0] = height[0];
    for (int i = 1; i < n; i++) leftMax[i] = Math.max(height[i], leftMax[i - 1]);

    rightMax[n - 1] = height[n - 1];
    for (int i = n - 2; i >= 0; i--) rightMax[i] = Math.max(height[i], rightMax[i + 1])

    int water = 0;
    for (int i = 0; i < n; i++) {
        water += Math.min(leftMax[i], rightMax[i]) - height[i];
    }
    return water;
}
```

4. The "Partition" Strategy (Dutch National Flag)

Theory: Use three pointers to move elements into three distinct zones.

Question: Sort Colors (0s, 1s, 2s).

Logic: low tracks the boundary for 0s, high tracks the boundary for 2s, and curr iterates.

```
public void sortColors(int[] nums) {
    int low = 0, curr = 0, high = nums.length - 1;
    while (curr <= high) {
        if (nums[curr] == 0) swap(nums, curr++, low++);
        else if (nums[curr] == 2) swap(nums, curr, high--); // Don't move curr! Re-check
        else curr++;
    }
}
```

Problem	Trick / Pattern	Core DS
Product Except Self	Prefix & Suffix Array	Single <code>int[]</code> result
Next Permutation	Pivot & Successor	Array pointers
Top K Frequent	Bucket Sort	<code>List[]</code> bucket
Kth Largest	Quickselect	Partition recursion
Contains Duplicate III	Bucket Windows	<code>HashMap</code> as buckets
Text Justification	Greedy + Space Dist	String arithmetic

Question Clue	Pattern	Performance Gain
"Range updates"	Difference Array	$O(M+N)$ instead of $O(M \cdot N)$
"Find missing/dupes"	State Flagging	$O(1)$ space instead of $O(N)$ hashset
"Global boundaries"	Two-Pass (L-R)	Resolves bilateral dependency
"3-way sorting"	Dutch Flag	$O(N)$ single-pass

When justifying text, remember:

Single word in line: Left-justify only.

Last line: Left-justify only.

Even distribution: Spaces between words should be distributed as evenly as possible. If it doesn't fit perfectly, give more spaces to the left slots.

HFT Revision Tip: "Cache Locality"

In HFT, 1D Arrays are king because they are stored contiguously in memory. This means the CPU can "prefetch" the next elements into the L1 cache.

Avoid: Large `ArrayList` (objects take more space and are scattered).

Prefer: Primitive `int[]` or `long[]`.

The "Two-Pass" cost: Even though $O(2N)$ is $O(N)$, doing two passes is often slower in HFT than a single complex pass because you have to load the data from memory into the cache twice.